THE
ONE
python™
vs
C#
WHY PYTHON WON AT MACHINE LEARNING?
BROADCASTING · NUMPY · PYTORCH · TRANSFORMER IN 3 LINES
.NET OXFORD
MARCH 2026

DANIEL BRYARS

- 20 years in .NET — C# by day, Python by night
- Recently completed MSc in AI (heavy Python use)

dotnetoxford-pythonvdotnet.bryars.com

slides and code available here

TWO LANGUAGES, TWO PHILOSOPHIES

PYTHON (1991)

Guido van Rossum

- Readability — “code is read more than written”
- Glue language — shell scripts and C
- A hobby project — Christmas 1989
- Bundled with Linux — on every server by 2000

C# (2000)

Anders Hejlsberg

- Java lawsuit — MS needed its own language
- C++ + Delphi — created Delphi
- Closed source — until 2014
- Windows-only — no Linux until .NET Core

TIMELINE

1991	PY	Python released
1995	PY	NumPy predecessor (Numeric)
~2000	PY	Bundled with Linux distros (Red Hat, Debian)
2000	C#	C# 1.0 (closed source, Windows-only)
2001	PY	SciPy & IPython — free Matlab alternative
2005	PY	NumPy 1.0 (wraps Fortran BLAS/LAPACK)
2007	PY	scikit-learn begins
2011	PY	IPython Notebook launches
2012	PY	Deep learning explodes (AlexNet)

more

TIMELINE

2014	C#	.NET goes open source / cross-platform
2014	PY	Jupyter Project (from IPython)
2015	PY	TensorFlow released (Google)
2016	PY	PyTorch released (Facebook/Meta)
2018	C#	ML.NET 1.0
2021	C#	TorchSharp (PyTorch for .NET)
2022	C#	Generic Math / INumber<T> (.NET 7)

PYTHON

- Dynamic
- ~100x slower
- No type safety

C#

- Strongly typed
- JIT-compiled
- Battle-tested tooling

Yet → Python dominates ML

ECOSYSTEM CONVERGENCE

- **NumPy** — Fortran BLAS/LAPACK under the hood
- **PyTorch / TensorFlow** — CUDA kernels
- **Jupyter** — interactive notebooks

Everything speaks **NumPy**

```
import numpy as np
from scipy import signal
import matplotlib.pyplot as plt

t = np.linspace(0, 1, 1000)                # NumPy
noisy = np.sin(2*np.pi*5*t) + np.random.randn(len(t))
filtered = signal.filtfilt(*signal.butter(4, 15, fs=1000), noisy)  # SciPy ←
plt.plot(t, filtered)                       # Matplotlib ← NumPy
```

One array type. Every library accepts it. No adapters needed.

Python doesn't do the maths — it orchestrates C, Fortran, and CUDA

```
# @ calls into BLAS dgemm (Fortran) — not Python
C = A @ B                          # matrix multiply

# Broadcasting — no loops, no copies
prices = np.array([10, 20, 30])  # shape (3,)
tax    = np.array([[1.1],
                   [1.2]])       # shape (2, 1)
totals = prices * tax            # shape (2, 3) — automatic

# Vectorised operations — one call to C
distances = np.sqrt(np.sum((points - centre)**2, axis=1))

# Slicing — views, not copies
batch = images[0:32, :, :, :]    # zero-cost slice
```

Succinct and powerful — but can trip you up (print shapes early and often)

JUPYTER VS LINQPAD

JUPYTER

- Fernando Pérez 2001 (as IPython)
- Browser-based notebooks
- Inline plots & rich output
- Standard in academia

LINQPAD

- Joseph Albahari 2007
- Desktop scratchpad for .NET
- Brilliant — but niche
- No academic adoption

Same idea, different worlds. Jupyter reached researchers. LINQPad reached developers.

940321122 Assessment2_Machine

localhost:8888/notebooks/940321122%20Assessment2_MachineLearning.ipynb

jupyter 940321122 Assessment2_MachineLearning Last Checkpoint: last year

File Edit View Run Kernel Settings Help Trusted

Markdown JupyterLab Python 3 (ipykernel)

```
batch_size = -1 # -1 turn off batching
learning_rate = 0.001
shuffle=False #Basically don't shuffle with periodic data

#For SDG optimiser
momentum=0.99
weight_decay = 0.001
```

Initialize the NN

[10]:
```
# initialize the NN
model = Net(n_hidden)
print(model)

def count_parameters(model):
    return sum(p.numel() for p in model.parameters() if p.requires_grad)

print(f'The model has {count_parameters(model):,} trainable parameters')
```

```
Net(
  (fc1): Linear(in_features=1, out_features=300, bias=True)
  (fc2): Linear(in_features=300, out_features=300, bias=True)
  (fc3): Linear(in_features=300, out_features=3, bias=True)
  (activation_function): Snake()
)
The model has 92,103 trainable parameters
```

Specify Loss Function and Optimizer

[11]:
```
#https://stats.stackexchange.com/questions/541277/how-can-i-get-the-phase-difference-between-two-frequencies
#https://stackoverflow.com/questions/6157791/find-phase-difference-between-two-inharmonic-waves
def phase_loss_tensor(predicted, target):
    predicted = predicted.cpu().detach().numpy()
    target = target.cpu().detach().numpy()
    return phase_loss(predicted, target)

def phase_loss(predicted, target):
    correlation = scipy.signal.correlate(predicted, target)
    return np.argmax(correlation) - (len(target) - 1)


criterion = nn.MSELoss()  # MSE Regression
#criterion = nn.L1Loss() # MAE (mean absolute error)
#criterion = nn.SmoothL1Loss()  # Huber Loss

#criterion = phase_loss_tensor

#optimizer = optim.Adam(model.parameters(), lr=learning_rate)  # Adam optimizer (no second order)
optimizer = optim.SGD(model.parameters(), lr=learning_rate, momentum=momentum, weight_decay=weight_decay)
```

Transfer to GPU if we've got one

[17]:
```
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print(device)
print(torch.__version__)

model = model.to(device)
#criterion = criterion.to(device)
```

940321122 Assessment2_Machine

localhost:8888/notebooks/940321122%20Assessment2_MachineLearning.ipynb

jupyter 940321122 Assessment2_MachineLearning Last Checkpoint: last year

File Edit View Run Kernel Settings Help Trusted

Markdown JupyterLab Python 3 (ipykernel)

```
    train_loss = 0.0
    for data, target in train_loader:
        data, target = data.to(device), target.to(device)
        optimizer.zero_grad()
        output = model(data)
        loss = criterion(output, target)
        loss.backward()
        optimizer.step()
        train_loss += loss.item() * data.size(0)

    train_loss = train_loss / len(train_loader.dataset)
    train_losses.append(train_loss)

    # Validate
    model.eval()
    val_loss = 0.0
    with torch.no_grad():
        for val_data, val_target in val_loader:
            val_data, val_target = val_data.to(device), val_target.to(device)
            val_output = model(val_data)
            val_loss += criterion(val_output, val_target).item() * val_data.size(0)

    val_loss = val_loss / len(val_loader.dataset)
    val_losses.append(val_loss)

    print(f'\rEpoch: {epoch+1} \tTraining Loss: {train_loss:.6f} \tValidation Loss: {val_loss:.6f}', end="")

runDescription = str(model)
runDescription += f"\n learning_rate:{learning_rate}"
runDescription += f"\n n_epochs:{n_epochs}"
runDescription += f"\n parameters:{count_parameters(model)}"
runDescription += f"\n criterion:{criterion}"
runDescription += f"\n optimizer:{optimizer}"
runDescription += f"\n device:{device}"
runDescription += f"\n train_loss:{train_loss}"

# Plot training and validation loss
fig, ax = plt.subplots(1, 1, figsize=(15, 15))
ax.plot(train_losses, label="Training Loss")
ax.plot(val_losses, label="Validation Loss", color='red', linestyle='dashed')
ax.set_title(f"{criterion.__class__.__name__} Loss vs Epoch")
ax.legend()
```

```
None
```

```
Epoch: 1000     Training Loss: 0.000198         Validation Loss: 1.293606
```

jupyter 940321122 Assessment2_MachineLearning Last Checkpoint: last year

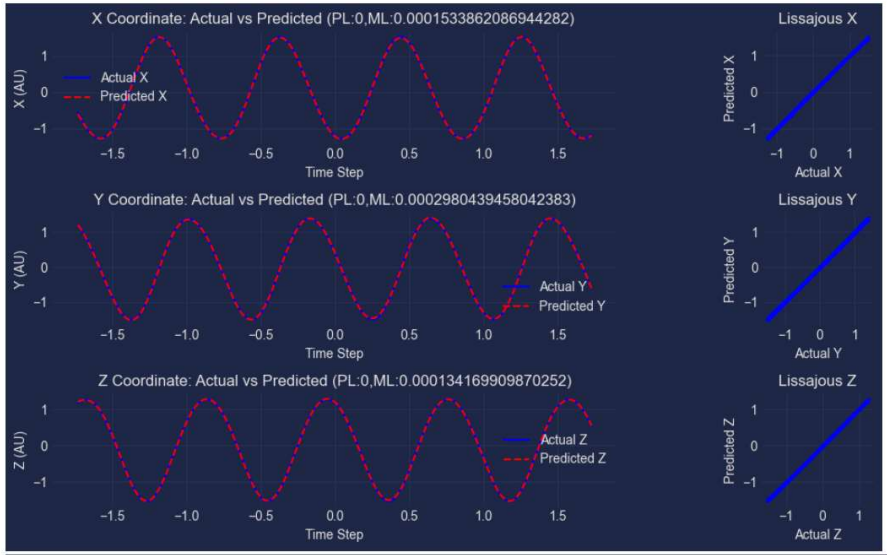
File Edit View Run Kernel Settings Help Trusted

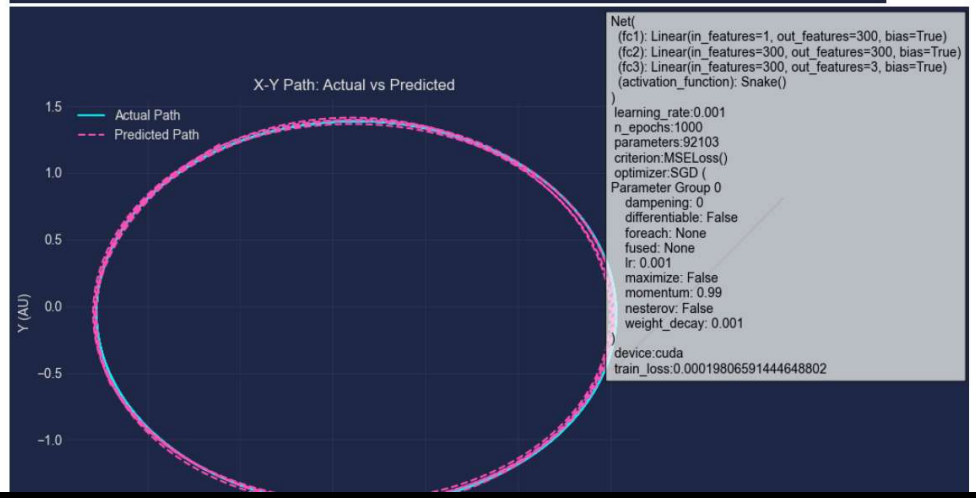
Markdown JupyterLab Python 3 (ipykernel)

```
    predicted_positions = model(inputs).cpu().numpy()

actual_positions = targets.cpu().numpy()

plotResults(inputs.cpu(),actual_positions,predicted_positions,runDescription)
```






```
Net(
  (fc1): Linear(in_features=1, out_features=300, bias=True)
  (fc2): Linear(in_features=300, out_features=300, bias=True)
  (fc3): Linear(in_features=300, out_features=3, bias=True)
  (activation_function): Snake()
)
learning_rate:0.001
n_epochs:1000
parameters:92103
criterion:MSELoss()
optimizer:SGD (
Parameter Group 0
    dampening: 0
    differentiable: False
    foreach: None
    fused: None
    lr: 0.001
    maximize: False
    momentum: 0.99
    nesterov: False
    weight_decay: 0.001
)
device:cuda
train_loss:0.00019806591444648802
```

TAKEAWAY

Python won because it made
fast things easy

C# can win where it makes
safe things fast

LIVE EXAMPLES

Example	Python	C#
GPU computing	`python gpu.py`	`dotnet run` (Gpu/)
Autograd	`python autograd.py`	`dotnet run` (Autograd/)
Train MLP (XOR)	`python mlp.py`	`dotnet run` (Mlp/)
Duck typing	`python duck_typing.py`	`dotnet run` (DuckTyping/)
Lorenz attractor	`python lorenz.py`	`dotnet run` (Lorenz/)
Ecosystem (NumPy→SciPy→plot)	`python ecosystem.py`	—
Vectorisation benchmark	`python vectorisation.py`	—

Python: `cd examples/python && pip install -r requirements.txt`
C#: `cd examples/csharp/<folder> && dotnet run`

DUCK TYPING

PYTHON

```python
def f(x):
    return x*x + 2*x

f(3)                        # 15
f(2.5)                      # 11.25
f(1+2j)                     # (-2+6j)
f(np.array([1,2,3]))        # [3, 8, 15]
f(torch.tensor([1.,2.,3.])) # GPU tensor
```

C#

```csharp
T F<T>(T x) where T : INumber<T>
{
    var two = T.One + T.One;
    return x * x + two * x;
}

F(3);       // int
F(2.5);     // double
// Complex?  won't compile
// Array?    need LINQ
// Tensor?   different API
```

*Python dynamically resolves the * and + operators at runtime — if the type supports them, it just works*

AUTOGRAD — CALCULUS FOR FREE

PYTHON

```python
import torch

x = torch.tensor(3.0, requires_grad=True)

y = x**2 + 2*x + 1
y.backward()

print(x.grad)  # 8.0
```

C#

```csharp
using TorchSharp;

var x = torch.tensor(3.0f,
    requiresGrad: true);

var y = x.pow(2) + 2 * x + 1;
y.backward();

Console.WriteLine(
    x.grad()!.item<float>());
```

`x**2` vs `x.pow(2)` • `x.grad` vs `x.grad()!.item<float>()` • Paper cuts add up

GPU COMPUTING

PYTHON

```python
import torch

a = torch.randn(1000, 1000)
b = torch.randn(1000, 1000)
c = a @ b           # CPU

a, b = a.cuda(), b.cuda()
c = a @ b           # GPU
```

C#

```csharp
using TorchSharp;

var a = torch.randn(1000, 1000);
var b = torch.randn(1000, 1000);
var c = a.mm(b);     // CPU

a = a.cuda();
b = b.cuda();
c = a.mm(b);         // GPU
```

TRAIN A NEURAL NETWORK (XOR)

PYTHON — 12 LINES

```python
import torch
import torch.nn as nn

X = torch.tensor([[0,0],[0,1],
    [1,0],[1,1]], dtype=torch.float)
y = torch.tensor([[0],[1],
    [1],[0]], dtype=torch.float)

model = nn.Sequential(
    nn.Linear(2, 8), nn.ReLU(),
    nn.Linear(8, 1), nn.Sigmoid())
opt = torch.optim.Adam(
    model.parameters(), lr=0.01)

for _ in range(1000):
    loss = nn.functional \
        .binary_cross_entropy(
            model(X), y)
    opt.zero_grad()
    loss.backward()
    opt.step()
```

C# — SAME THING?

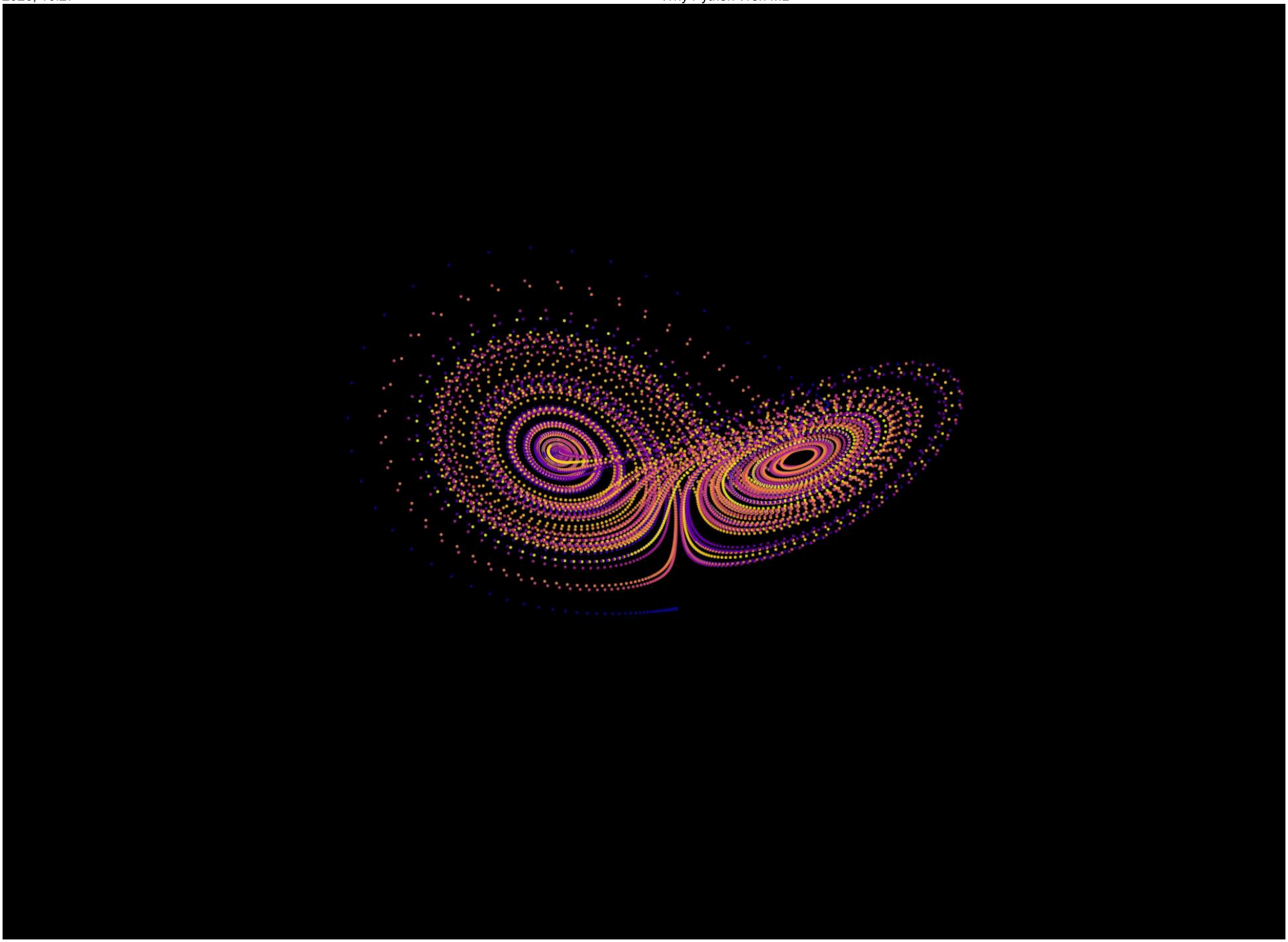
```csharp
using TorchSharp;
using static TorchSharp.torch;
using static TorchSharp.torch.nn;

var X = torch.tensor(new float[,]
    {{0,0},{0,1},{1,0},{1,1}});
var y = torch.tensor(new float[,]
    {{0},{1},{1},{0}});

var model = Sequential(
    ("fc1", Linear(2, 8)),
    ("relu", ReLU()),
    ("fc2", Linear(8, 1)),
    ("sig", Sigmoid()));
var opt = torch.optim.Adam(
    model.parameters(),
    learningRate: 0.01);

for (int epoch = 0; epoch < 1000;
     epoch++) {
    using var pred =
        model.forward(X);
    using var loss = functional
        .binary_cross_entropy(
            pred, y);
    opt.zero_grad();
    loss.backward();
    opt.step();
}
```

Named tuples • `using var` *for every tensor* • `model.forward(X)` *vs* `model(X)` • `learningRate:` *vs* `lr=`



LORENZ ATTRACTOR

PYTHON

```python
import numpy as np
import matplotlib.pyplot as plt

dt, n = 0.01, 10000
sigma, beta, rho = 10, 8/3, 28
xyz = np.zeros((n, 3))
xyz[0] = [0.1, 0, 0]

for i in range(n - 1):
    x, y, z = xyz[i]
    xyz[i+1] = xyz[i] + dt*np.array([
        sigma*(y-x),
        x*(rho-z) - y,
        x*y - beta*z])

fig = plt.figure(facecolor="black")
ax = fig.add_subplot(projection="3d")
ax.scatter(*xyz.T,
    c=np.arange(n), cmap="plasma")
```

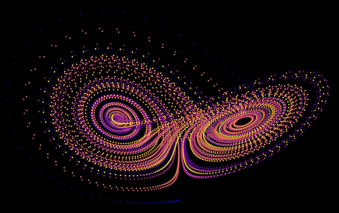
C#

```csharp
const double dt = 0.01;
const int n = 10000;
const double sigma = 10,
    beta = 8.0/3, rho = 28;

var x = new double[n];
var y = new double[n];
var z = new double[n];
x[0] = 0.1;

for (int i = 0; i < n-1; i++) {
    x[i+1] = x[i]+dt*sigma*(y[i]-x[i]);
    y[i+1] = y[i]+dt*(x[i]*(rho-z[i])-y[i]);
    z[i+1] = z[i]+dt*(x[i]*y[i]-beta*z[i]);
}

// Plot? ScottPlot? OxyPlot?
// None do 3D scatter well.
// Export to CSV, open in Python.
```



ECOSYSTEM — EVERYONE SPEAKS NUMPY

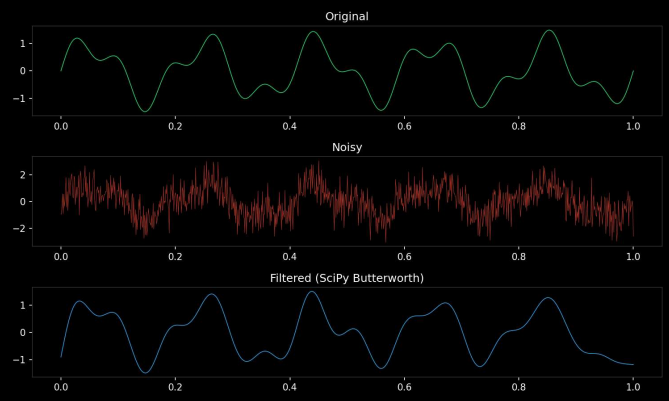
PYTHON

```python
import numpy as np
from scipy import signal
import matplotlib.pyplot as plt

t = np.linspace(0, 1, 1000)
noisy = np.sin(2*np.pi*5*t) \
      + np.random.randn(len(t))

b, a = signal.butter(4, 15, fs=1000)
filtered = signal.filtfilt(b, a, noisy)

plt.plot(t, filtered)
plt.show()
```

C#

```csharp
// Step 1: generate signal
//   Math.NET? System.Numerics?
// Step 2: filter it
//   Math.NET.Filtering? DIY?
// Step 3: plot it
//   ScottPlot? OxyPlot?
//
// Three libraries.
// Three different array types.
// Three sets of docs.
//
// Or...
//
// Process.Start("python",
//     "ecosystem.py");
```